Project Planning and Scope Management

Scope management ensures the project includes all the work required, and only the work required, to complete it successfully. We turn requirements into a scope statement, a work breakdown structure and a scope baseline that change control protects.

PMBOK Guide 7th ed. | Scrum Guide 2020 | PRINCE2 7 | Case study: Realtime Quiz

AGENDA

What we will cover

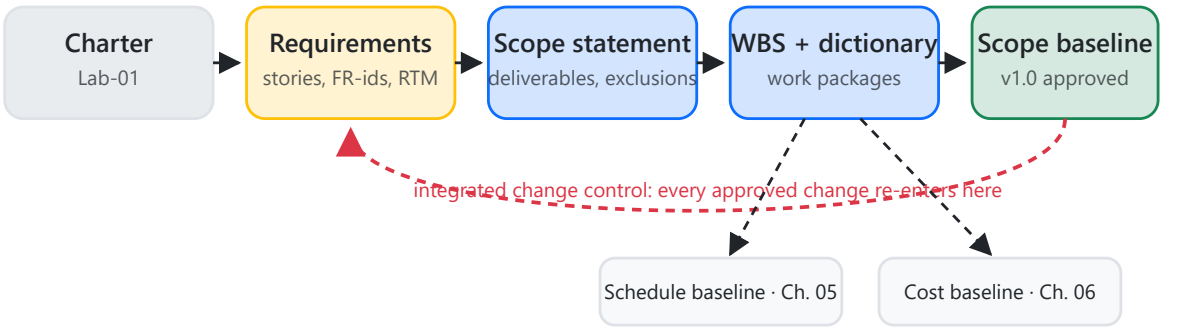
1. The project management plan and its subsidiary plans
2. Collecting requirements: interviews, workshops, prototypes, user stories
3. Requirements documentation and the traceability matrix
4. Project scope statement: deliverables, acceptance criteria, exclusions
5. Product scope versus project scope
6. Work breakdown structure: decomposition rules, 100 percent rule, work packages
7. WBS dictionary and control accounts
8. Scope baseline and integrated change control
9. Validating scope: acceptance and sign-off
10. Product backlog and story mapping as agile scope tools
11. Chapter Quiz
12. Lab-04

Click any item to jump directly to that topic.

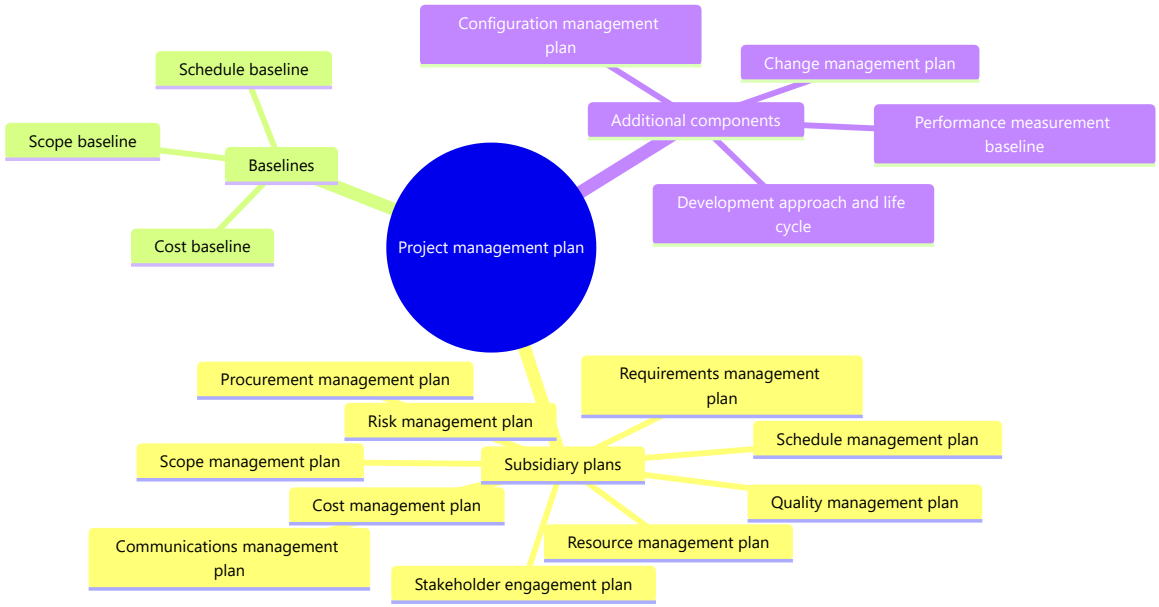
From charter to scope baseline: the planning chain of this chapter

- Chapter 01 gave us a **charter** (Lab-01): why the Realtime Quiz project exists, its high-level scope and its sponsor, the Head of the IT department.
- Chapter 02 showed that scope drives time and cost; Chapter 03 chose a **hybrid life cycle**: predictive milestones with two-week sprints.
- Now we make scope precise: **requirements** (what stakeholders need), a **scope statement** (what we will deliver and will not), a **WBS** (how the work is broken down) and a **WBS dictionary**.
- Together these three documents form the **scope baseline**. Chapters 05 and 06 build the schedule and the budget on it.
- The baseline is protected by **integrated change control**: scope changes are decided, not discovered.

: Guide 7th ed., Planning performance domain · Process Groups Practice Guide: Plan Scope, Collect Requirements, Define Scope, Crea



The project management plan: one document, many subsidiary plans



- The **project management plan** says *how* the project will be executed, monitored, controlled and closed. It is not the schedule; the schedule is one of its baselines.
- Each knowledge area contributes a **subsidiary plan**: the scope management plan defines how scope is defined, validated and controlled; the requirements management plan defines how requirements are collected, prioritised and traced.
- The three **baselines** (scope, schedule, cost) are the approved versions the project is measured against.
- In this course each lab produces one subsidiary plan in its own folder; Lab-12 assembles them into the final plan.
- A plan is **progressively elaborated**: the version 1.0 you write now will be revised through change control, never silently overwritten.

> 💡 PMBOK Guide 7th ed. treats planning as a performance domain; the subsidiary plans above come from the Process Groups Practice Guide (2022). PRINCE2 7 calls the equivalent the Project Initiation Documentation (PID).

Worked example: the scope management plan of the Realtime Quiz project

```
## 4. Scope management plan (v1.0, Lab-04)

### 4.1 How scope is defined
- Requirements come from the charter, the Realtime Quiz plan
  (Must / Should / Could) and two stakeholder workshops.
- Must features become user stories US-01..US-18 with
  Given/When/Then acceptance criteria.
- The scope statement lists deliverables, acceptance criteria,
  exclusions, constraints and assumptions.

### 4.2 How the WBS is built and maintained
- Deliverable-oriented, 100 percent rule, codes 1..7 at level 2.
- Work packages: 8 to 80 hours, one owner, one dictionary entry.
- The WBS lives in `lab04/wbs.csv`; the mindmap is generated.

### 4.3 How scope is validated
- Each sprint review: sponsor checks deliverables against the
  acceptance criteria; sign-off recorded in
  `scope-validation-checklist.md`.

### 4.4 How scope is controlled
- Any request to change the baseline uses form CR-nnn.
- Under 8 h: PM decides. 8 h or more: CCB (sponsor, PM, dev lead).
- Approved changes bump the baseline version (v1.0 -> v1.1).
```

Question the plan answers	Where the answer lives
How do we collect and prioritise requirements?	Requirements management plan (4.1)
How do we build and code the WBS?	WBS rules (4.2)
Who accepts a deliverable, and how?	Validate scope procedure (4.3)
Who may change the baseline?	Change control procedure (4.4)
Where is the current baseline?	`scope-statement.md`, `wbs.csv`, `wbs-dictionary.md`, all tagged `scope-v1.0`

⚠ **Common mistake:** writing the scope management plan as a copy of the scope. The plan describes the *process* (who, how, when); the scope statement describes the *content*.

Collecting requirements: choose the elicitation technique for the question you have

Technique	Best when	Output	Effort	Pitfall	Realtime Quiz use
Interview	Few key stakeholders, deep knowledge	Notes, quotes, goals	Low per person	Interviewer bias, one opinion	Head of IT: pilot goals, constraints
Facilitated workshop	Conflicting views need consensus	Agreed feature list, priorities	Medium, all at once	Loud voices dominate	Lecturers agree MoSCoW of features
Focus group	Learn how a user group thinks	Themes, preferences	Medium	Group-think	8 students on the play page
Questionnaire	Many users, simple questions	Statistics	Low per answer	No follow-up questions	Devices and browsers of 200 students
Observation	Users cannot describe their work	Real workflow, pain points	High	Observer changes behaviour	Watch a Kahoot session in class
Prototype	UI or novel features are unclear	Feedback on a mock-up	Medium	Mistaken for the product	Clickable host console in draw.io
Document analysis	Rules already written down	Constraints, data	Low	Outdated documents	Grading policy, GDPR-style data rules
Benchmarking	Competitors set expectations	Feature comparison	Low	Copying instead of deciding	Kahoot, Mentimeter feature scan
User stories	Agile delivery, evolving detail	Backlog items with criteria	Continuous	Stories without acceptance criteria	US-01..US-18 in Lab-04 Task 1

- **Requirement**: a condition or capability that must be present in the product to satisfy a stakeholder need. Kinds: business, stakeholder, solution (functional, non-functional), transition, project, quality.
- Start with the **stakeholder register** (Lab-01) to decide whom to ask; mix at least two techniques so one source cannot mislead you.
- Prioritise with **MoSCoW** (Must, Should, Could, Won't): the Realtime Quiz plan already does this, so the MVP is the Must list.
- Record every requirement with an **id** (FR-A1, FR-S6): the traceability matrix depends on stable ids.

Worked example, interview guide (5 questions): 1. What happens in class today? 2. What goes wrong? 3. What must the pilot prove? 4. What would make you refuse to use it? 5. Who else should we ask?

User stories: a card, a conversation and a confirmation in Gherkin

````
### US-11 · Submit one answer while the question is open
**As a** student in a live session
**I want** to submit exactly one answer before the deadline
**so that** my score is fair and cannot be changed after the fact.

Priority: Must   Source: FR-S6, FR-S7   Size: 5 points
WBS: 3.2 Live session engine

Acceptance criteria (Gherkin):
```gherkin
Scenario: answer accepted before the deadline
  Given question 3 is open with a 30 second limit
  When I submit option B after 10 seconds
  Then I receive ANSWER_ACK with accepted = true
  And the host sees answeredCount increased by 1

Scenario: second answer is rejected
  Given I already answered question 3
  When I submit option C
  Then I receive ANSWER_ACK with accepted = false
  And the reason is "already answered"

Scenario: late answer is rejected
  Given question 3 was closed by the timer
  When I submit option B
  Then I receive ANSWER_ACK with accepted = false
  And the reason is "question closed"
```
````

INVEST	Check on US-11
Independent	Needs an open question (US-09) but no other story
Negotiable	UI of the confirmation is open; the rule is not
Valuable	Fair scoring is the point of the product
Estimable	Team sized it at 5 points
Small	Fits in one sprint
Testable	Three scenarios become three JUnit tests

- **Card**: the one-line story. **Conversation**: details agreed with the product owner. **Confirmation**: the acceptance criteria.
- **Given / When / Then** states the precondition, the action and the observable result; a criterion without an observable result cannot be validated.
- Stories describe the *product*. Project work (CI pipeline, pilot, documentation) goes into the WBS, not into user stories for the sponsor.

⚠ "The system shall be fast" is not testable. "Answer acknowledged within 300 ms on the LAN" (the plan's NFR) is.

Requirements documentation: every requirement carries the same attributes

Attribute	Why it is there
Id	Stable key for traceability (never reused)
Statement	One requirement, one sentence, active voice
Type	Functional, non-functional, transition, project
Source	Stakeholder or document it came from
Priority	MoSCoW; decides the MVP
Acceptance criterion	How we will know it is met
Verification	Test, inspection, demonstration, analysis
Status	Proposed, approved, implemented, verified, deferred

- A good requirement is **unambiguous, complete, consistent, testable, traceable** and **necessary**.
- The requirements document plus the traceability matrix are outputs of Collect Requirements (Process Groups Practice Guide); they are inputs to the scope statement and to Chapter 07 test planning.

```
REQUIREMENTS REGISTER (excerpt)  · lab04/requirements.csv
id     type  statement                                src     pri verify  status
FR-A1  FR    Guest registers with email, name,      plan    M   test    approved
             password and role
FR-S1  FR    Instructor opens a session; server     plan    M   test    approved
             issues a unique 6-character code
FR-S6  FR    One answer per participant per         plan    M   test    approved
             question, only while open
FR-C1  FR    points * (0.5 + 0.5 * remaining/limit) plan    M   test    approved
NFR-1  NFR   Broadcast reaches all clients          plan    M   demo    approved
             within 500 ms on the LAN
FR-R4  FR    Results downloadable as CSV and PDF    plan    S   test    approved
FR-N3  FR    Session chat, instructor can mute      plan    C   test    deferred
TR-1   trans Instructors trained in one 45-minute   Head IT M   inspect proposed
             session before the pilot
PR-1   proj  Pilot with 50 students in week 14      charter M   demo    approved

Rule: id + one sentence + one verification method. If a row needs
two verification methods, it is two requirements.
```

The requirements traceability matrix: requirement → story → WBS work package → test

Requirement	User story	WBS work package	Test (Ch. 07)	Status
FR-A1 register with role	US-01	2.3 Accounts and JWT security	AuthControllerTest.registerStudent	verified
FR-A2 login returns JWT, 12 h	US-02	2.3 Accounts and JWT security	JwtServiceTest.expiresAfter12h	verified
FR-Q1 create quiz	US-04	4.1 Quiz and question API	QuizServiceTest.createQuiz	implemented
FR-Q3 question rules (2-6 options, 5-300 s)	US-05	4.1 Quiz and question API	QuestionValidationTest	implemented
FR-S1 open session, 6-char code	US-08	3.2 Live session engine	SessionServiceTest.openGeneratesCode	approved
FR-S3 join by code, live lobby	US-09	3.2, 3.4	SessionStompTest.lobbyBroadcast	approved
FR-S6 one answer while open	US-11	3.2 Live session engine	ConcurrencyTest.oneAnswerPerParticipant	approved
FR-C1 speed-based points	US-12	3.3 Scoring and leaderboard	ScoringServiceTest.speedBonus	approved
FR-S8 leaderboard top 10 + own rank	US-13	3.3 Scoring and leaderboard	LeaderboardTest.top10PlusSelf	approved
FR-R4 CSV and PDF export	US-16	5.3 CSV and PDF export	ResultControllerTest.csvContentType	proposed
FR-S9 reconnect keeps score	none	none	none	**gap**
none	none	6.3 Production deployment	smoke test on gicitc.com	project work, fine

Spreadsheet check: `=COUNTIF(RTM!C:C, "3.2*")` counts stories per work package; a Must requirement whose story column is empty is a scope gap; a work package that appears nowhere delivers either project work (acceptable) or nothing (delete it).

- **Forward traceability**: from each requirement to the work that delivers it and the test that proves it. Answers "is everything covered?"
- **Backward traceability**: from each work package or test back to a requirement. Answers "why are we doing this?" and exposes gold plating.
- The matrix is a **living document**: statuses move from proposed to verified; new rows only through change control.
- The red row is the point of the exercise: FR-S9 is a Must in the plan, but nobody wrote a story for it. Found now it costs one story; found in the pilot it costs the pilot.

Keep the matrix as `traceability-matrix.csv` in Git next to the WBS; review it in the same merge request as any scope change.

The project scope statement: what we deliver, how it is accepted, what we will not do

```
# Project scope statement · Realtime Quiz platform (v1.0)

## 1. Product scope description
A web application for live classroom quizzes: an instructor authors
multiple-choice quizzes, opens a live session with a 6-character join
code and drives it question by question; students answer within a time
limit and see instant feedback, points and a leaderboard. Built on
Spring Boot 4.1.1 with STOMP over WebSocket and PostgreSQL 17.

## 2. Project deliverables
D1  Deployed application on the gicitc.com VPS (HTTPS, Docker Compose)
D2  Source repository with CI (GitHub Actions, `./mvnw verify` green)
D3  Automated test suite, 80 % line coverage on service and realtime
D4  User guide for instructors and students (Markdown, 10 pages max)
D5  UML documentation set (11 diagrams) and final report
D6  Pilot session with 50 students in week 14, results exported
D7  Project management plan with all subsidiary plans (this course)

## 3. Acceptance criteria (per deliverable, checked by the sponsor)
D1  Health endpoint returns UP; a session with 50 participants runs
    for 10 questions with no error in the server log.
D3  JaCoCo report shows >= 80 % lines; CI status badge green on main.
D6  At least 45 of 50 invited students submit at least one answer;
    CSV export of the pilot session handed to the Head of IT.
```

Section	Purpose
Product scope description	Elaborates the charter's high-level description with the requirements collected
Deliverables	Unique, verifiable products or results, including project management ones
Acceptance criteria	Conditions that must be met before the sponsor accepts a deliverable
Exclusions	What is explicitly out, so nobody assumes it is in
Constraints	Limits the team must work within (time, budget, technology)
Assumptions	Facts taken as true without proof; each one is a risk (Ch. 09)

- The charter says *why* and roughly *what*; the scope statement says precisely *what* and *what not*.
- Deliverables are **nouns** ("user guide"), never activities ("write documentation").
- Each acceptance criterion names a **measurement** and a **judge** (here: the Head of IT).

Exclusions, constraints and assumptions: the half of the scope statement that prevents disputes

```
## 4. Project exclusions (out of scope for version 1.0)
- Native mobile apps (the responsive web UI is sufficient)
- Question types other than single- and multi-answer multiple choice
- Video conferencing or screen sharing
- Integration with an LMS (Moodle, Canvas); CSV export covers it
- Horizontal scaling across several server nodes
- Admin screen and shared question bank (Could features, deferred)
- Content: the team writes no quiz questions for the lecturers

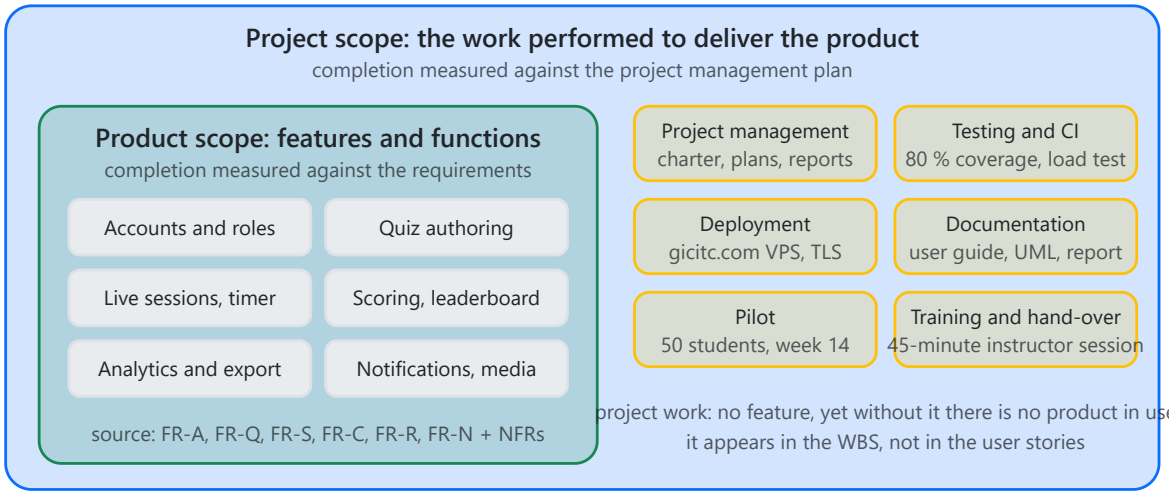
## 5. Constraints
- 14 teaching weeks; pilot fixed in week 14 (semester calendar)
- Team: 4 students, about 10 h each per week, plus the lecturer as PM
- Cash budget: VPS 25 USD/month, one domain, no paid tools
- Technology: Spring Boot 4.1.1, JDK 25, PostgreSQL 17 (course stack)
- Security: no personal data beyond email and display name

## 6. Assumptions
- The campus network allows WebSocket, or SockJS fallback works
- The 50 pilot students bring a phone or laptop with a modern browser
- The Head of IT is available for a 30-minute review every two weeks
- The VPS at gicitc.com stays available for the whole semester
```

- **Exclusions** come from the plan's Non-Goals and from every "could you also..." heard in the workshops. Writing them down costs a minute; arguing about them in week 13 costs a sprint.
- **Constraints** are given; you plan around them. Time and cash come from the charter (Lab-01), the technology from the Software Engineering course.
- Each **assumption** turns into a risk in Lab-09: "if the campus proxy blocks WebSocket, students cannot play" already has a mitigation in the plan (SockJS fallback).
- Review exclusions with the sponsor and get a **signature**: the scope statement is only useful when both sides have agreed to it.

⚠ **Common mistake:** an exclusion list that only repeats the deliverables in the negative. Good exclusions are things a reasonable stakeholder *might* expect, such as writing the quiz content or training every lecturer.

Product scope versus project scope: the features, and all the work to deliver them

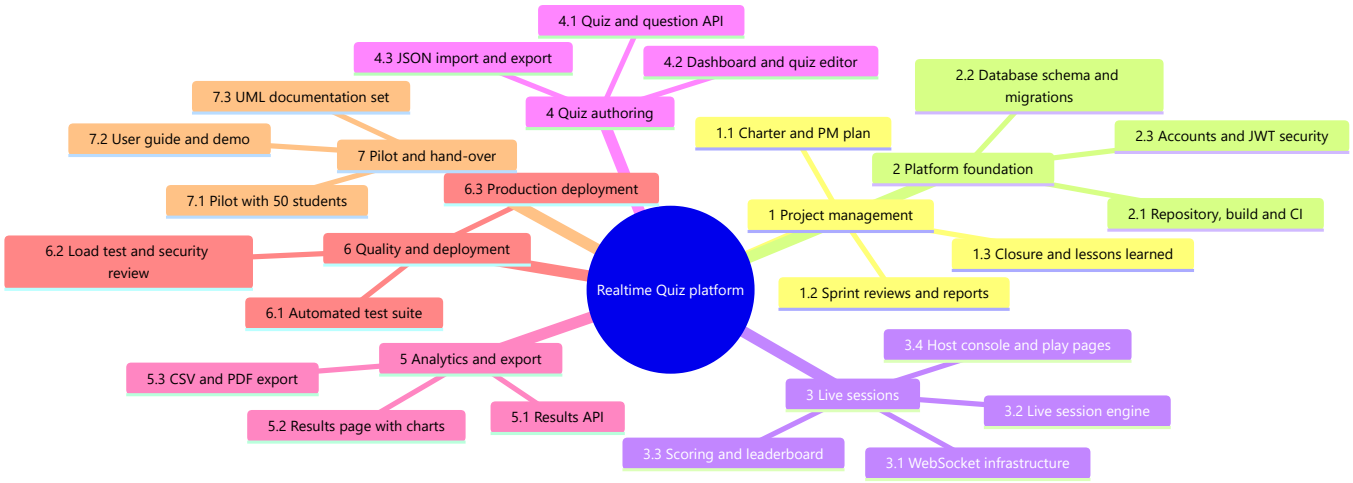


- **Product scope**: the features and functions that characterise the product, service or result. Verified against the *requirements*.
- **Project scope**: the work performed to deliver a product with those features. Verified against the *project management plan*. It always contains the product scope and more.
- Rule of thumb: if the sponsor can *use* it, it is product scope; if the team must *do* it so the sponsor can use the product, it is project scope.
- In the Realtime Quiz numbers: product work packages 3.x, 4.x, 5.x and 2.2-2.3 total 380 h; project-only packages (1.x, 2.1, 6.x, 7.x) total 220 h, about 37 % of 600 h. Forgetting them is the most common estimation error.

💡 **Scope creep** adds product scope without approval; **gold plating** adds features nobody asked for. Both increase project scope silently and are caught by backward traceability.

TOPIC 6

The work breakdown structure: a hierarchical decomposition of the total scope of work



- The **WBS** organises and defines the total scope; work not in the WBS is outside the project (PMBOK Guide).
- **Deliverable-oriented**: every element is a noun (a thing that exists when done), never a verb phrase. "3.3 Scoring and leaderboard", not "implement scoring".
- Level 1 is the project, level 2 the major deliverables or phases, the lowest level the **work packages** that can be estimated, scheduled and assigned to one owner.
- **100 percent rule**: each level contains 100 % of the work of the level above, no more and no less. It applies to the parent, to the children and to every level.
- A mind map is the fastest way to *draft* a WBS; the outline with codes on the next slide is how you *publish* it.

Seven level-2 branches, 22 work packages, 600 h in total. Deliberately mirrors the plan's phases and deliverables so Chapter 05 can schedule them.

The same WBS as an outline with codes, and the rules for decomposing it

```
Realtime Quiz platform                 600 h
1  Project management                   60 h
   1.1 Charter and PM plan              20 h
   1.2 Sprint reviews and reports       30 h
   1.3 Closure and lessons learned      10 h
2  Platform foundation                  80 h
   2.1 Repository, build and CI         20 h
   2.2 Database schema and migrations   20 h
   2.3 Accounts and JWT security        40 h
3  Live sessions                       160 h
   3.1 WebSocket infrastructure         30 h
   3.2 Live session engine              60 h
       3.2.1 Session state machine
       3.2.2 Question timer
       3.2.3 STOMP controller and events
       3.2.4 Concurrency tests
   3.3 Scoring and leaderboard          30 h
   3.4 Host console and play pages      40 h
4  Quiz authoring                      100 h
5  Analytics and export                 60 h
6  Quality and deployment               80 h
7  Pilot and hand-over                  60 h
```

Code `3.2.4`: level 4, child of 3.2, grandchild of 3. Codes are stable: deleting 3.3 does not renumber 3.4.

Rule	Meaning	Realtime Quiz check
100 percent rule	Children sum to the parent, in scope and in effort	3.1 + 3.2 + 3.3 + 3.4 = 30 + 60 + 30 + 40 = 160 h = branch 3
Mutually exclusive	No work appears in two elements	Leaderboard is in 3.3 only, not also in 3.4
Deliverable nouns	Outcomes, not activities	"Automated test suite", not "do testing"
8/80 rule	A work package takes 8 to 80 hours	3.2 at 60 h passes; a 120 h package would be split
Reporting period rule	No package longer than one reporting period	Two-week sprint of 4 x 20 h = 80 h fits
One owner	Each work package has exactly one accountable person	3.2: realtime lead; 3.4: frontend lead
Stop rule	Stop when you can estimate, schedule, assign and verify	3.2.x are activities for Ch. 05, not WBS elements

- Decompose by **deliverable** (this WBS), by **phase** (Setup, Build, Pilot) or by **subsystem** (backend, frontend, ops); pick one principle per level and do not mix them at the same level.
- Rolling wave: distant branches (7 Pilot) stay coarse now and are decomposed when their sprint approaches; mark them as *planning packages*.

Worked example: checking branch 3 against the 100 percent rule, and finding the gap

```
FIRST DRAFT of branch 3 "Live sessions" (estimate for branch: 160 h)

  code  work package                  hours  requirements covered
  3.1   WebSocket infrastructure        30   NFR-1 latency, STOMP auth
  3.2   Live session engine             50   FR-S1 S2 S4 S5 S6 S7 S11
  3.3   Scoring and leaderboard         30   FR-C1 C2 C3 C4, FR-S8
  3.4   Host console and play pages     35   FR-S3 (lobby), play page
  ------------------------------------------------------------
  sum of children                      145   vs parent 160  -> 15 h ?

Check 1 (effort):  160 - 145 = 15 h of the branch is not in any child.
Check 2 (scope):   branch 3 must cover FR-S1..S11 and FR-C1..C4.
                   FR-S9 reconnect and resume ..... covered by: none
                   FR-S10 remove participant ..... covered by: none
                   -> two Must requirements have no work package.

FIX (v1.0 of the WBS)
  3.2   Live session engine        50 + 10 = 60 h   adds FR-S9 (resume)
  3.4   Host console and play pages 35 + 5 = 40 h   adds FR-S10 (remove)
  sum of children  30 + 60 + 30 + 40 = 160 h = parent   OK
  every FR-S and FR-C id appears in exactly one dictionary entry   OK
```

- Apply the rule **twice**: once on effort (do the children add up?) and once on scope (is every requirement assigned to exactly one child?). Matching sums alone prove nothing: 15 missing hours could have hidden inside a padded estimate.
- The **traceability matrix** is the tool for the scope check: filter the Must rows and look for empty WBS cells.
- The fix changes the work packages, so their **dictionary entries** and the RTM change too. Baseline everything together as `scope-v1.0`.
- Decide who *owns* a requirement that spans two packages: FR-S3 (join) is implemented in 3.2 (server) and 3.4 (lobby page); the dictionary of each says which part.

⊗ **Anti-pattern:** adding a work package called "Other" or "Miscellaneous" to make the sums match. It has no deliverable, no acceptance criterion and no owner, so it cannot be validated.

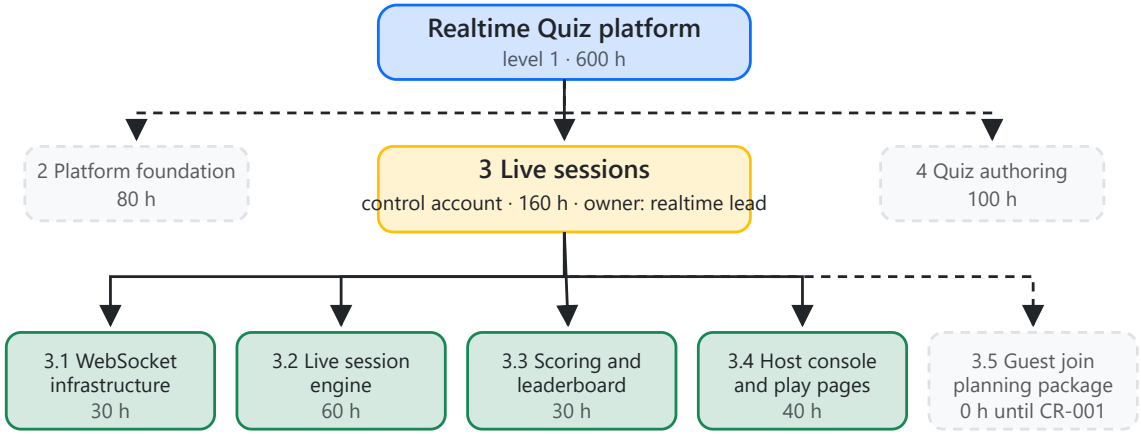
The WBS dictionary: one entry per work package turns a label into a contract

Field	3.2 Live session engine
Code / name	3.2 Live session engine (control account 3 Live sessions)
Description	Server-side engine that holds live session state in memory (SessionRegistry), moves a session through LOBBY, QUESTION_OPEN, QUESTION_CLOSED, FINISHED, enforces the deadline with one scheduled task per open question, and publishes every event through SessionEventPublisher.
Deliverables	SessionService, SessionRegistry, QuestionTimer, SessionStompController, STOMP event catalogue in the plan, unit tests including the 200-thread concurrency test
Requirements	FR-S1, FR-S2, FR-S4, FR-S5, FR-S6, FR-S7, FR-S9, FR-S11; NFR timer accuracy (drift under 200 ms)
Acceptance criteria	Two sessions on the same quiz run in parallel without interference; a late answer after deadline + 250 ms is rejected; 200 concurrent submits yield exactly one answer per participant; a reconnecting participant keeps their score; all tests green in CI.
Owner	Realtime lead (student C); reviewer: backend lead
Effort / cost	60 h internal; 0 USD cash
Schedule	Sprints 3 and 4 (weeks 5 to 8); milestone: MVP checkpoint demo end of week 8
Dependencies	Predecessors 2.2, 2.3, 3.1; successor 3.3, 3.4
Risks / notes	Late-answer race (per-session lock, see plan); memory growth from abandoned sessions (2-hour eviction job)

- The WBS names the element; the **dictionary** says what it contains, how it is accepted, who owns it and what it costs. Without it, "3.2" means something different to every reader.
- Requirements listed here are the **link to the RTM**; acceptance criteria here are what the sponsor will check in Validate Scope.
- Effort and schedule fields are filled progressively: rough now, refined in Chapters 05 and 06. The dictionary version follows the baseline version.
- **PRINCE2 7** uses the same idea under a different name: product-based planning yields a product breakdown structure and a *product description* per product, with quality criteria and quality method.

Lab-04 Task 3 asks for five dictionary entries. Write the acceptance criteria first: if you cannot state how the package will be accepted, it is not yet a work package.

Control accounts and planning packages: where scope, schedule and cost are measured together



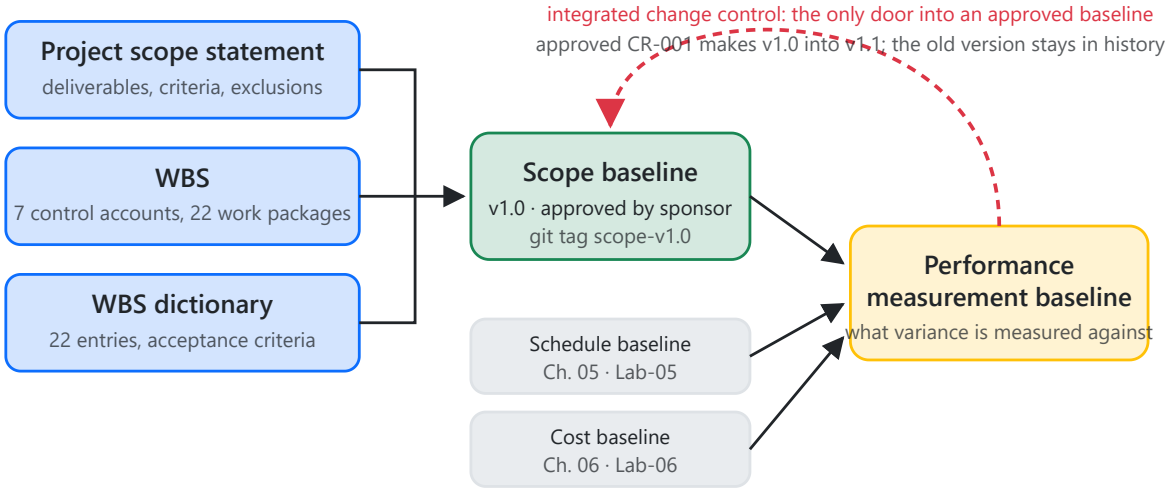
work packages: estimated, scheduled, assigned · sum = 160 h = control account

earned value (Ch. 06) is reported per control account: PV, EV and AC roll up from its work packages

- A **control account** is a management control point where scope, budget and schedule are integrated and compared with earned value. Usually a level-2 or level-3 element; one control account may contain several work packages, each work package belongs to exactly one control account.
- A **planning package** sits below a control account with known scope and budget but without detailed activities yet (rolling wave). 3.5 stays a dashed box until change request CR-001 is approved.
- Control accounts define the **reporting granularity**: the Head of IT sees seven lines, the team sees 22 work packages.
- Naming the control account owner now gives Chapter 08 its RACI rows and Chapter 06 its cost accounts for free.

💡 The 22 work packages of the Realtime Quiz WBS roll up into 7 control accounts of 60, 80, 160, 100, 60, 80 and 60 h: 600 h in total.

The scope baseline: three approved documents that only change control may touch

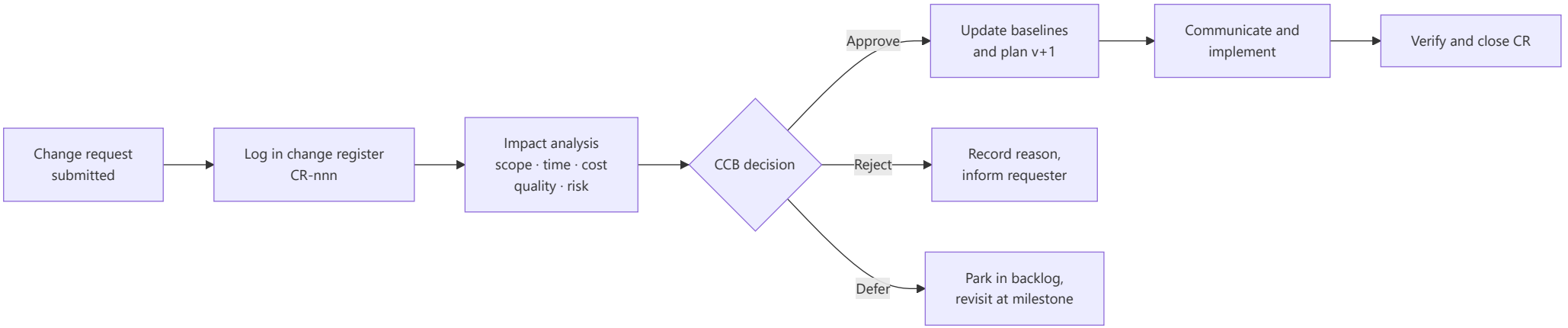


- A **baseline** is the approved version of a work product, changed only through formal change control and used as the basis for comparison to actual results.
- The **scope baseline** = approved scope statement + WBS + WBS dictionary. Nothing else; the RTM and the backlog support it but are not baselined.
- Scope, schedule and cost baselines together form the **performance measurement baseline** (PMB) that earned value (Ch. 06) measures against.
- In Git: commit the three files, tag `scope-v1.0`, and record the sponsor's approval date in `lab04/README.md`. A new approved change means a new tag, never a force push.

⚠ Working from an unapproved draft is the classic way to lose an argument in week 13. "Baselined" means signed, dated and versioned.

TOPIC 8

Integrated change control: request → impact analysis → CCB decision → update the baselines



- **Anyone** may request a change; only the **change control board** (CCB) may approve one that touches a baseline. For the Realtime Quiz: sponsor (Head of IT), the PM (lecturer) and the dev lead.
- **Impact analysis** is done before the decision, by the team, on all constraints: scope, time, cost, quality, risk, stakeholders. "Small change" is a conclusion, not an input.
- Three outcomes: **approve** (baselines and plan are updated, version incremented), **reject** (reason recorded, requester informed), **defer** (parked, revisited at the next milestone).
- **Thresholds** keep the board useful: under 8 h the PM decides and reports; 8 h or more, or any change to a deliverable or acceptance criterion, goes to the CCB within five working days.

💡 **Agile does not abolish change control**: within a sprint the scope is fixed (Scrum Guide 2020: only the Product Owner may change the Sprint Backlog with the developers); between sprints the backlog is re-ordered. The baseline is the release scope, not each sprint.

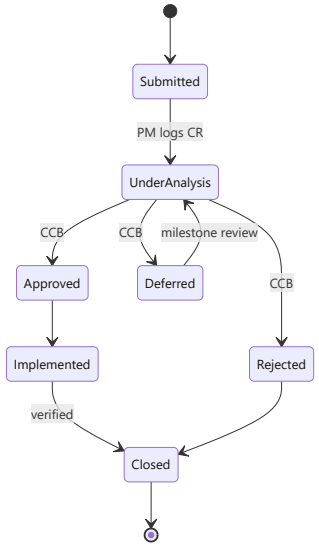
⚠ Scope creep is not the change request you approved; it is the ten small ones nobody logged.

Worked example: change request CR-001 "Add anonymous joining with a nickname"

```
CHANGE REQUEST CR-001                          Status: APPROVED (deferred start)
Title         Add anonymous joining with a nickname (no account)
Requester     Head of IT (sponsor), after the sprint 2 review, week 5
Reason        Two lecturers want to run quizzes in open-day events where
              visitors have no account. Listed as "Could" in the plan.
Description   A session flag allowGuests; guests join with code + nickname,
              receive a 24-hour guest token; nickname unique per session.
Requirements FR-S3 amended (guest may join if allowed); new FR-S12.

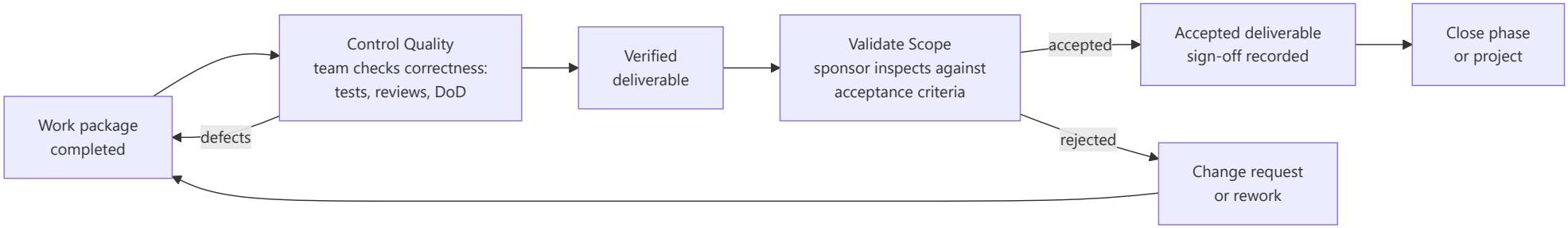
IMPACT ANALYSIS (by the team, 2 h)
  Scope       new work package 3.5 Guest join; dictionary entry; RTM +2 rows
  Time        16 h; fits the stretch slot in sprint 5 (weeks 10-11);
              pilot date (week 14) unchanged; no critical-path impact
  Cost        +16 h internal (from the 40 h management reserve); 0 USD cash
  Quality     +1 item in the security checklist (rate limit 10 joins/min/IP,
              guest cannot access analytics); +3 unit tests
  Risk        abuse of open sessions (new risk R-12, medium); mitigated by
              rate limit and the instructor's remove-participant control
  Alternatives  (a) reject: lecturers create throw-away accounts
                (b) defer to v1.1 after the pilot

CCB DECISION  2026-10-30 · Head of IT, PM, dev lead
  Approved, on condition that work starts only after the MVP checkpoint
  demo (end of week 8). Scope baseline v1.0 -> v1.1; WBS adds 3.5 (16 h);
  branch 3 becomes 176 h, project 616 h; reserve 40 h -> 24 h.
  Communicated to team and lecturers on 2026-10-31.
```



- The form makes the 100 percent rule hold after the change: the new package gets hours, a dictionary entry and RTM rows, and every parent total is updated.
- A change request that does not name where the hours come from (reserve, de-scoping, more time) is not ready for the board.

Validating scope: the sponsor formally accepts what quality control has already verified



	Control Quality (Ch. 07)	Validate Scope (this chapter)
Question	Is it built correctly?	Is it what we agreed to deliver?
Who	The team, QA lead	Sponsor or customer
Against	Quality requirements, Definition of Done, tests	Scope baseline: acceptance criteria in the scope statement and dictionary
When	Continuously, before validation	At the end of each sprint, phase or deliverable
Output	Verified deliverable, defect list	Accepted deliverable, signed record, or change request

- **Inspection** is the technique: the sponsor reviews, measures and tests the deliverable against the written criteria; the demo at a sprint review is exactly this.
- Acceptance is **formal**: a dated record with the criterion, the evidence and the signature. Verbal "looks good" is not acceptance.
- A rejected deliverable produces either **rework** (the criterion was not met) or a **change request** (the criterion itself must change). Never both silently.
- Accepted deliverables are the input to **closing** the phase or the project (Ch. 03).

Worked example: scope validation record for the sprint 1 deliverables

```
# Scope validation record · Sprint 1 review · 2026-10-16

Deliverables: 2.1 Repository, build and CI · 2.2 Schema and migrations
              2.3 Accounts and JWT security · 4.1 Quiz and question API

| # | WBS | Acceptance criterion            | Evidence           | Result   |
|---|-----|---------------------------------|--------------------|----------|
| 1 | 2.1 | `./mvnw verify` green on main   | Actions run #41    | Accepted |
| 2 | 2.2 | V1..V3 apply on PostgreSQL 17   | Flyway log, demo   | Accepted |
| 3 | 2.3 | register, login, JWT 12 h;      | Postman demo,      | Accepted |
|   |     | wrong password gives 401        | 6 unit tests       |          |
| 4 | 4.1 | 2-6 options, 5-300 s enforced   | 400 lacks a field  | Rejected |
|   |     | with field-level errors         | error for `points` |          |

Decision on #4: rework, not a change request. The criterion stands;
`points` validation (1-10000) is added in sprint 2. Re-inspect 2026-10-30.

Accepted by: _____________________  Head of IT          Date: 2026-10-16
Presented by: _____________________  Project manager     Date: 2026-10-16
```

Term	Meaning	Caught by
Scope creep	Uncontrolled growth of product or project scope without adjusting time, cost or resources	Change log, backward traceability
Gold plating	Delivering extras nobody asked for (an animated leaderboard when a list was agreed)	Validate Scope against the criteria, not against effort
Partial acceptance	Accepting deliverables one by one, with a record for each	Sprint reviews, this record
Conditional acceptance	Accepted with a listed, dated defect to fix	Only with a rework date and an owner

- Row 4 shows the difference between *quality* and *scope*: the API exists (scope delivered) but is not correct (quality). It goes back to Control Quality, not to the CCB.
- Keep every record in `lab04/scope-validation-checklist.md`; the closure report (Ch. 03) is the list of all accepted deliverables.

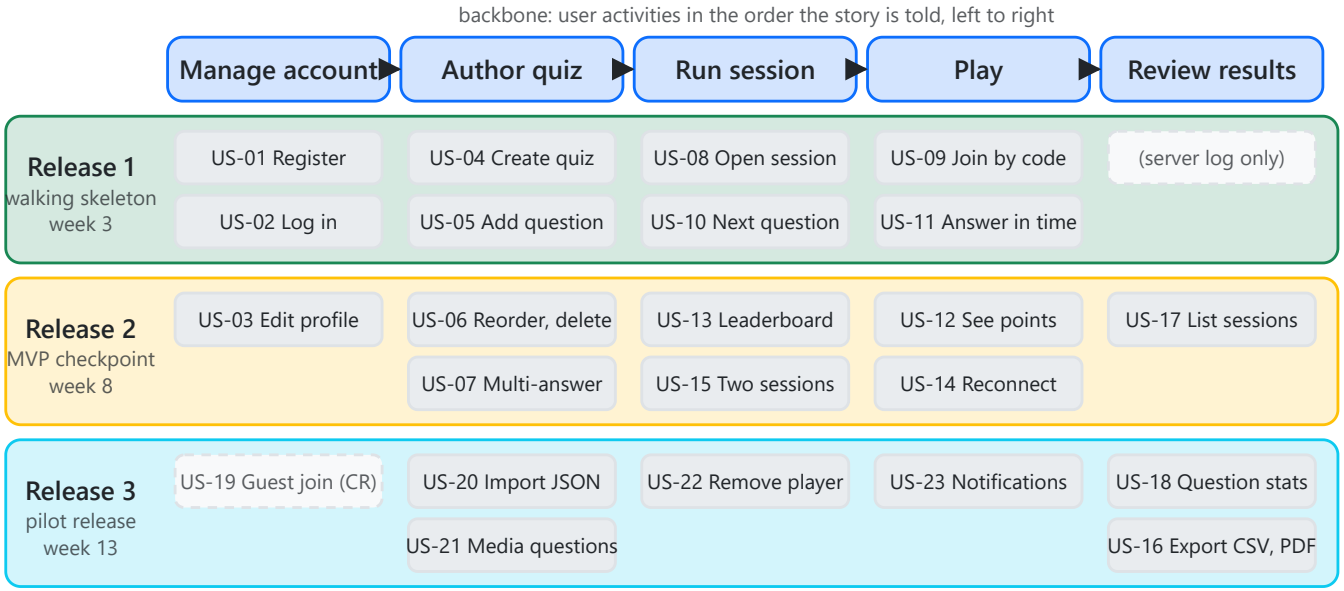
The product backlog: an ordered, emergent list of what the product needs

```
PRODUCT BACKLOG (excerpt, ordered)      Product Goal: run one real quiz
                                        with 50 students in week 14
rank  id     story (short)                    pri  pts  WBS   sprint
  1   US-01  Register with role                M    3   2.3   1
  2   US-02  Log in, receive JWT               M    3   2.3   1
  3   US-04  Create quiz with title            M    3   4.1   1
  4   US-05  Add question with 2-6 options     M    5   4.1   1
  5   US-08  Open session, get join code       M    5   3.2   2
  6   US-09  Join by code, see lobby           M    5   3.2   2
  7   US-10  Advance: start, next, close, end  M    8   3.2   2
  8   US-11  Submit one answer while open      M    5   3.2   3
  9   US-12  Points by speed (FR-C1)           M    3   3.3   3
 10   US-13  Leaderboard top 10 + own rank     M    3   3.3   3
 11   US-14  Reconnect and keep score          M    5   3.2   4
 12   US-16  Export results CSV and PDF        S    5   5.3   5
 13   US-19  Guest join with nickname (CR-001) C    5   3.5   5
 ...
velocity so far: 18 points per sprint -> ranks 1-4 fit sprint 1 (14 pts)
```

	WBS (predictive)	Product backlog (agile)
Organised by	Deliverable hierarchy	Value order (rank 1 first)
Completeness	100 % of the work, fixed at baseline	Emergent; refined every sprint (DEEP: detailed appropriately, estimated, emergent, prioritised)
Unit	Work package, hours	Product Backlog item, story points
Owner	Project manager with the team	Product Owner (Scrum Guide 2020)
Change	Change control board	Re-ordering by the Product Owner; sprint scope fixed
Covers project work?	Yes (PM, deployment, pilot)	Only if made explicit (enablers, spikes)

- In the hybrid life cycle of Lab-03 both coexist: the WBS is the **baseline** the sponsor signs; the backlog is how the team **sequences** product work packages into sprints. The WBS column keeps them consistent.
- A Must story with no WBS code, or a work package with no story and no project deliverable, is a defect in one of the two.
- The **Definition of Done** (Scrum Guide 2020) is the agile form of Control Quality; the sprint review is the agile form of Validate Scope.

Story mapping: the backbone of user activities, the tasks beneath, and the release slices across



- A **story map** (Jeff Patton) arranges stories in two dimensions: the **backbone** of user activities left to right in narrative order, and **user tasks** beneath each activity, most essential on top.
- Horizontal **release slices** cut across every activity: each release is a thin, end-to-end usable version, not one finished column.
- The **walking skeleton** (release 1) proves the whole flow in week 3: register, create a quiz, open a session, join, answer. Nothing is polished, everything connects.
- The map is the sponsor-friendly view of the same scope as the WBS: every card carries a story id, every story has a WBS code, so the two reconcile.

A flat backlog hides the gaps; the map shows them: the dashed "Review results" cell in release 1 says the instructor has no results view until release 2, a deliberate decision, now visible.

Best practices and common mistakes in scope management

1 Ids everywhere

FR-S6, US-11, 3.2, CR-001: stable identifiers make the traceability matrix, the dictionary and the change log one connected system. Never reuse or renumber an id.

2 Nouns in the WBS

"Automated test suite", not "testing". A WBS element is something that exists when the work is done; the verbs belong to the activity list in Chapter 05.

3 Check the 100 percent rule twice

Effort: children sum to the parent. Scope: every Must requirement is in exactly one work package. Matching hours with a missing requirement is still a broken WBS.

4 Write exclusions people might expect

"No LMS integration", "no quiz content written by the team", "no native app". An exclusion nobody would have assumed is wasted ink; one that someone assumed saves a sprint.

5 Acceptance criteria before estimates

If you cannot say how the sponsor will accept a work package, you cannot estimate it either. Fill the dictionary's acceptance field first, the hours second.

6 Baseline means signed and tagged

Sponsor approval date in the plan, Git tag `scope-v1.0`, new tag per approved change. A draft on someone's laptop is not a baseline.

Mistake: "it is a small change"

Size is the output of impact analysis, not a reason to skip it. Log every request; the ten unlogged small ones are the scope creep that sinks week 13.

Mistake: confusing quality with scope

A buggy API is a Control Quality problem (rework); a missing API is a Validate Scope problem (change request or gap). Route them to the right process.

Mistake: gold plating as goodwill

An animated leaderboard nobody asked for costs hours, adds defects and gives the sponsor nothing they can validate. Propose it as a change request or leave it.

Chapter quiz 10 questions

1. Which document is NOT part of the scope baseline?

- Project scope statement
- Work breakdown structure
- WBS dictionary
- Requirements traceability matrix

2. Branch "3 Live sessions" is estimated at 160 h. Its work packages are 30 + 50 + 30 + 35 h. What does the 100 percent rule tell you?

- Nothing, estimates may differ by 10 %
- 15 h of the branch is not in any work package: decompose further or fix the estimates
- The parent estimate must be lowered to 145 h
- Add a work package named Other with 15 h

3. In this deck (project = level 1, "3 Live sessions" = level 2), at which level is WBS code 3.2.4?

- Level 2
- Level 3
- Level 4
- Level 5

4. Running the pilot with 50 students and writing the user guide belong to...

- Product scope only
- Project scope but not product scope
- Neither, they are out of scope
- The requirements traceability matrix

5. Which elicitation technique lets users react to a working mock-up before the real product exists?

- Questionnaire
- Document analysis
- Prototype
- Benchmarking

6. Who performs Validate Scope, and against what?

- The team, against the Definition of Done
- The sponsor or customer, against the acceptance criteria of the scope baseline
- The CCB, against the change log
- The QA lead, against the test plan

7. A stakeholder asks for a free-text question type, which the scope statement lists as an exclusion. What is the correct route?

- Implement it if a developer has spare time
- Refuse: exclusions can never change
- Log a change request, run an impact analysis, let the CCB decide
- Add it to the sprint backlog and inform the sponsor later

8. The RTM lists 24 Must requirements; 21 of them have a work package in the WBS column. How many forward-traceability gaps are there?

- 0, the WBS is complete
- 3
- 21
- 24

9. A proposed work package is estimated at 120 h. Under the 8/80 rule you should...

- Accept it, it is below one sprint of team capacity
- Split it into two or more work packages of 8 to 80 h each
- Merge it with a neighbouring package
- Convert it into a control account with no children

10. In a story map, what do the horizontal slices across all backbone activities represent?

- Releases: thin end-to-end usable versions of the product
- Departments of the organisation
- WBS control accounts
- Test levels from unit to end-to-end

WRAP-UP

Summary

The project management plan is a set of subsidiary plans plus three baselines; the scope management plan describes the process, not the content.

Collect requirements with at least two techniques, prioritise with MoSCoW, give every requirement a stable id and a verification method.

User stories carry Given/When/Then acceptance criteria; the traceability matrix links requirement, story, work package and test and exposes gaps.

The scope statement lists deliverables, acceptance criteria, exclusions, constraints and assumptions; exclusions prevent the disputes of week 13.

Product scope is the features, measured against requirements; project scope is all the work, measured against the plan, about a third of it non-product.

The WBS is deliverable-oriented, coded, obeys the 100 percent rule in effort and in scope, and stops at work packages of 8 to 80 hours with one owner.

The WBS dictionary turns each work package into a contract; control accounts are where scope, schedule and cost meet for earned value.

Scope statement + WBS + dictionary = scope baseline, versioned and changed only by a change request, impact analysis and a CCB decision.

Control Quality verifies correctness; Validate Scope is the sponsor's formal, recorded acceptance. Backlogs and story maps are the agile views of the same scope.

Open Lab-04: 5 tasks + 1 challenge

Next chapter: 05 · Project Scheduling and Time Management